

커뮤니티 댓글 기능 & 데이터베이스 구조 안내서

게시판/댓글/좋아요 기능이 데이터베이스 상에서 어떻게 구성되어 있고, 이번
작업 세션에서 무엇이 새로 만들어졌는지를 개발 배경 지식이 없어도 이해할 수
있도록 정리한 문서입니다.

프로젝트: KDT_TravelPlanner

다루는 범위: 커뮤니티 게시판 DB 구조 전체 + 이번 세션에서 추가된 댓글/좋아요/수정 기능

작성일: 2026-08-24

1. 이 문서는 무엇을 다루나요

먼저 큰 그림부터 — 커뮤니티 게시판이 무엇으로 이루어져 있는지

이 서비스에는 사용자들이 자유롭게 글을 쓰고 서로 댓글을 남길 수 있는 **커뮤니티 게시판**이 있습니다. 게시판은 크게 네 가지 요소로 이루어져 있습니다.

- 1. **게시글** — 사용자가 작성하는 글 한 편. 제목, 본문, 카테고리(여행후기/자유게시판/질문답변/공지사항), 태그를 가집니다.
- 2. **댓글** — 게시글 아래에 달리는 짧은 글. 대댓글(댓글의 댓글)은 없고 한 단계로만 존재합니다.
- 3. **좋아요(반응)** — 게시글이나 댓글에 누를 수 있는 하트 버튼. 같은 사용자가 같은 글에 두 번 좋아요를 누를 수 없습니다.
- 4. **카테고리 / 태그** — 게시글을 분류하는 탭(카테고리)과 자유롭게 붙이는 꼬리표(태그)입니다.

이번 작업 세션에서는 이 중 **댓글 기능을 처음부터 끝까지 새로 만들었습니다**. 댓글을 쓰고, 보고, 지우는 기본 기능은 물론, 이후 사용자 요청에 따라 **댓글 좋아요와 댓글 수정** 기능까지 추가했습니다. 이 문서는 그 과정에서 만들어진 데이터베이스 구조 전체와, 화면에서 실제로 어떻게 동작하는지를 함께 설명합니다.

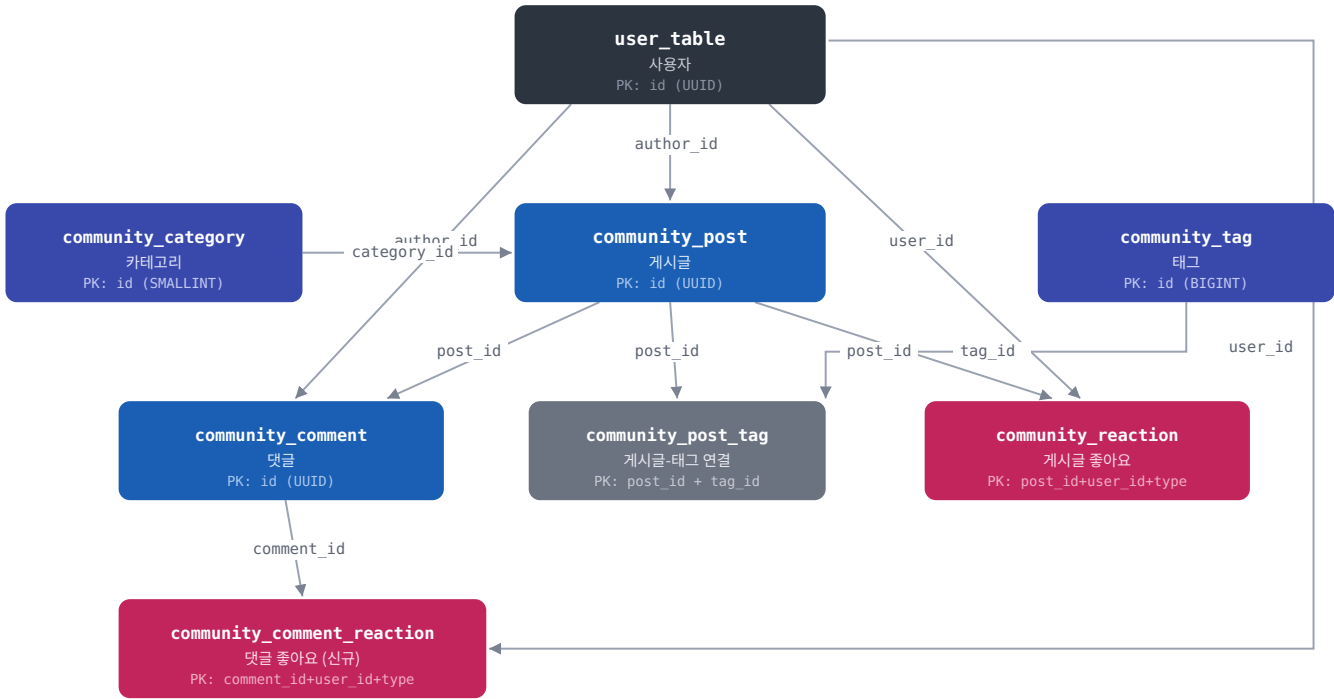
🔗 읽기 전에 알아두면 편한 용어

- 데이터베이스(DB) 테이블** — 엑셀 시트라고 생각하면 됩니다. 각 행(row)이 데이터 하나(게시글 하나, 댓글 하나 등), 각 열(column)이 그 데이터의 속성(제목, 작성자, 작성일 등)입니다.
- UUID** — 데이터마다 붙는 고유한 식별번호입니다. 1,2,3 처럼 순서대로 매기지 않고 무작위 문자열(예: 3f2a9c1e-...)을 씁니다. 순번을 보고 "전체 게시글이 몇 개인지" 유추하지 못하게 하려는 목적도 있습니다.
- 외래키(FK, Foreign Key)** — "이 데이터가 저 테이블의 어떤 행과 연결되어 있다"는 표시입니다. 예를 들어 댓글 테이블의 `author_id` 는 "이 댓글을 쓴 사람이 사용자 테이블의 누구인지"를 가리킵니다.
- 소프트 삭제(Soft Delete)** — 실제로 데이터를 지우지 않고 "삭제된 시각"만 기록해 화면에서만 안 보이게 하는 방식입니다. 이렇게 하면 실수로 지웠을 때 복구가 쉽고, 나중에 통계를 낼 때도 유리합니다. 이 프로젝트의 게시글/댓글은 모두 이 방식을 씁니다.
- CASCADE(연쇄 삭제)** — 원본 데이터가 지워지면 그것과 연결된 데이터도 함께 자동으로 지워지도록 하는 규칙입니다. 예를 들어 사용자가 탈퇴하면 그 사람이 쓴 댓글도 함께 정리됩니다.

2. 데이터베이스 구조

커뮤니티 게시판을 이루는 8개 테이블과 그 관계

커뮤니티 게시판 데이터는 총 8개의 테이블에 나뉘어 저장됩니다. 아래 그림은 각 테이블이 서로 어떤 컬럼으로 연결되어 있는지를 보여줍니다. 화살표는 "왼쪽/위 테이블의 데이터를 참조한다"는 뜻입니다.



파란색 계열 = 이번 세션 이전에 이미 있던 테이블 · 분홍색 = 좋아요(반응) 테이블 · 회색 = 연결 전용 테이블 · **community_comment_reaction**이 이번 세션에 새로 추가된 테이블입니다.

⚠ 이번 세션에 실제로 바뀐 것

이번 세션에서 데이터베이스 구조에 실제로 손을 댄 부분은 **딱 두 가지**입니다.

- 1) **community_comment_reaction** 테이블을 새로 만들었습니다(댓글 좋아요 저장용).
- 2) **community_comment** 테이블에 **updated_at** 컬럼 하나를 추가했습니다(댓글 수정 시각 기록용, 처음 값은 비어있음).

나머지 6개 테이블(**user_table** , **community_category** , **community_tag** , **community_post** , **community_post_tag** , **community_reaction**)은 이전 세션에서 이미 만들어져 있던 것이고, 이번 세션에서는 건드리지 않았습니다.

2-1. 테이블별 상세 설명

user_table — 사용자

서비스에 로그인한 사용자 정보. 게시글·댓글·좋아요는 모두 결국 이 테이블의 한 사용자와 연결됩니다.

컬럼	타입	설명
id	UUID (PK)	사용자 고유번호
provider	VARCHAR(20)	로그인 수단. GOOGLE 또는 NAVER 만 허용
provider_user_id	VARCHAR(255)	구글/네이버가 부여한 사용자 식별값. provider와 조합해서 유일해야 함
email / name / nickname	VARCHAR	이메일, 실명, 별명(별명은 서비스 내에서 유일)
profile_image_url	TEXT	프로필 사진 주소
gender / birth_year	VARCHAR / SMALLINT	선택 입력 프로필 정보
profile_completed	BOOLEAN	가입 후 필수 정보를 다 채웠는지 여부
created_at / updated_at / deleted_at	TIMESTAMP TZ	가입/수정/탈퇴(소프트 삭제) 시각

community_category — 카테고리

게시판 상단 탭. 현재 4개(여행후기/자유게시판/질문답변/공지사항)가 미리 등록되어 있습니다.

컬럼	타입	설명
id	SMALLINT (PK)	1~4의 작은 정수
code	VARCHAR(30)	화면 API에서 쓰는 영문 코드. 예: TRAVEL_REVIEW , FREE , QNA , NOTICE
name	VARCHAR(50)	화면에 보이는 한글 이름(예: "여행후기")
sort_order	INT	탭이 나열되는 순서
is_active	BOOLEAN	탭을 노출할지 여부

community_tag — 태그

게시글에 붙이는 자유 태그(예: "제주도", "혼행"). 사용자가 새 태그 이름을 입력하면 서버가 없는 태그일 때 자동으로 만듭니다.

컬럼	타입	설명
id	BIGINT (PK, 자동증가)	태그 고유번호
name	VARCHAR(20)	태그 이름. 중복 불가, 1~20자

community_post — 게시글

커뮤니티에 올라오는 글 본문 자체.

컬럼	타입	설명
id	UUID (PK)	게시글 고유번호
author_id	UUID (FK → user_table)	작성자. 작성자가 탈퇴하면 게시글도 함께 삭제(CASCADE)
category_id	SMALLINT (FK → community_category)	어느 카테고리(탭)의 글인지
title	VARCHAR(200)	제목. 빈 값 불가
body_json	JSONB	본문. 굵게/기울임/이미지 등 서식이 있는 리치텍스트 편집기(Tiptap)가 만든 구조화된 데이터
body_preview	VARCHAR(200)	목록 화면에 보이는 요약(본문 앞부분 120자)
source_travel_id	UUID (FK, nullable)	여행 일정을 바탕으로 후기를 쓴 경우, 원본 일정 ID. 일정이 지워지면 이 값만 비워짐
view_count	INT	조회수
version	INT	글 수정 시 다른 사람이 동시에 고치는 것을 막기 위한 버전 번호(낙관적 락)
created_at / updated_at / deleted_at	TIMESTAMPTZ	작성/수정/삭제 시각

community_post_tag — 게시글·태그 연결

게시글 하나에 태그를 여러 개 붙일 수 있고, 태그 하나가 여러 게시글에 쓰일 수도 있기 때문에(다대다 관계) 둘을 연결만 해주는 테이블입니다.

컬럼	타입	설명
post_id	UUID (FK, PK 일부)	게시글
tag_id	BIGINT (FK, PK 일부)	태그

게시글 아래 달리는 댓글. 대댓글 없이 한 단계로만 존재합니다.

컬럼	타입	설명
id	UUID (PK)	댓글 고유번호
post_id	UUID (FK → community_post)	어느 게시글의 댓글인지. 게시글이 지워지면 댓글도 함께 삭제(CASCADE)
author_id	UUID (FK → user_table)	작성자. 작성자가 탈퇴하면 댓글도 함께 삭제(CASCADE)
content	TEXT	댓글 내용. 빈 값(공백만 있는 값 포함) 불가
created_at	TIMESTAMP TZ	작성 시각
updated_at 신규	TIMESTAMP TZ (nullable)	수정 시각. 한 번도 수정한 적 없으면 비어있음(NULL) — 화면에서 "(수정됨)" 표시 여부를 이 값으로 판단
deleted_at	TIMESTAMP TZ (nullable)	삭제(소프트) 시각. 값이 있으면 실제로는 남아있지만 화면에는 안 보임

community_reaction — 게시글 좋아요

게시글에 대한 좋아요 기록. 지금은 "LIKE" 한 종류만 있지만, 나중에 다른 반응(예: 웃겨요 등)을 추가할 수 있도록 type 컬럼을 미리 열어두었습니다.

컬럼	타입	설명
post_id	UUID (FK, PK 일부)	좋아요가 눌린 게시글
user_id	UUID (FK, PK 일부)	좋아요를 누른 사용자
type	VARCHAR(20), PK 일부	반응 종류. 현재는 LIKE 만 허용

세 컬럼을 합쳐서 기본키(PK)로 쓰기 때문에, 같은 사용자가 같은 게시글에 같은 종류의 좋아요를 두 번 누르는 것 자체가 데이터베이스 차원에서 원천적으로 불가능합니다.

community_comment_reaction — 댓글 좋아요 이번 세션 신규 테이블

댓글에 대한 좋아요 기록. community_reaction과 구조는 똑같지만, 게시글 좋아요와 댓글 좋아요를 서로 다른 테이블로 분리해서 관리하기 위해 별도로 만들었습니다.

컬럼	타입	설명
comment_id	UUID (FK, PK 일부)	좋아요가 눌린 댓글. 댓글이 지워지면 좋아요 기록도 함께 삭제(CASCADE)
user_id	UUID (FK, PK 일부)	좋아요를 누른 사용자
type	VARCHAR(20), PK 일부	반응 종류. 현재는 LIKE 만 허용(체크 제약)

3. API 엔드포인트 (화면과 서버가 대화하는 창구)

브라우저 화면이 서버에 데이터를 요청/전송할 때 사용하는 주소 목록

 API가 뭔가요

화면(프론트엔드)은 데이터베이스에 직접 접근하지 않습니다. 대신 정해진 "주소(엔드포인트)"로 서버에 요청을 보내고, 서버가 데이터베이스를 확인한 뒤 결과를 돌려줍니다. GET 은 "조회", POST 는 "새로 만들기", PATCH 는 "일부 수정", PUT 은 "덮어쓰기/토글", DELETE 는 "삭제"를 뜻합니다.

Method	주소(Path)	설명	누가 할 수 있나
GET	/api/v1/community/categories	카테고리(탭) 목록 조회	누구나
GET	/api/v1/community/tags	태그 자동완성 후보 조회	누구나
GET	/api/v1/community/posts	게시글 목록 조회(검색/정렬/페이지네이션)	누구나
POST	/api/v1/community/posts	게시글 작성	로그인한 사용자
GET	/api/v1/community/posts/{postId}	게시글 상세 조회(조회수 자동 증가)	누구나
PATCH	/api/v1/community/posts/{postId}	게시글 수정	작성자 본인만
DELETE	/api/v1/community/posts/{postId}	게시글 삭제(소프트)	작성자 본인만
GET	/api/v1/community/posts/{postId}/comments	댓글 목록 조회	누구나
POST	/api/v1/community/posts/{postId}/comments	댓글 작성	로그인한 사용자
PATCH	/api/v1/community/comments/{commentId}	댓글 수정 신규	작성자 본인만
DELETE	/api/v1/community/comments/{commentId}	댓글 삭제(소프트)	작성자 본인만
PUT	/api/v1/community/posts/{postId}/reactions/{type}	게시글 좋아요 켜기/끄기(토글)	로그인한 사용자
PUT	/api/v1/community/comments/{commentId}/reactions/{type}	댓글 좋아요 켜기/끄기(토글) 신규	로그인한 사용자

 원래 설계 문서 범위 밖 확장

이 프로젝트에는 docs/community-api-contract.md 라는, 화면팀과 서버팀이 미리 합의한 "API 설계 계약서"가 있습니다. 그런데 이번에 추가한 댓글 수정과 댓글 좋아요 API, 그리고 댓글 응답에 새로 생긴 reactionCount (좋아요 수)· isReacted (내가 좋아요 눌렀는지)· updatedAt (수정 시각) 항목은 이 계약서에는 원래 없던 내용입니다. 기능상 필요해서 그때그때 확장한 것이라, 나중에 계약 문서도 이 구현에 맞춰 함께 업데이트하는 것을 추천합니다.

4. 기능이 실제로 어떻게 동작하나요

사용자 입장에서 화면을 조작했을 때 뒤에서 벌어지는 일

4-1. 댓글 작성 · 조회 · 삭제 (기본 기능)

게시글 상세 화면 하단에 댓글 목록과 댓글 입력창이 있습니다.

1. 페이지를 열면 화면이 `GET /posts/{postId}/comments` 로 그 게시글의 댓글을 모두 가져와 최신순(작성순)으로 보여줍니다.
2. 로그인한 사용자에게만 댓글 입력창이 보입니다. 로그인하지 않은 사용자에게는 "로그인하면 댓글을 남길 수 있어요" 안내와 로그인 버튼이 대신 보입니다.
3. 입력창에 글을 쓰고 **등록** 버튼을 누르면 `POST /posts/{postId}/comments` 로 서버에 전송되고, 성공하면 목록 맨 아래에 새 댓글이 바로 추가됩니다.
4. 각 댓글에는 삭제 버튼이 있는데, **본인이 쓴 댓글에만** 보입니다. 이는 클라이언트가 판단하는 게 아니라 서버가 댓글마다 내려주는 `isMine` 값을 그대로 신뢰해서 결정합니다(더 안전한 방식).
5. 삭제 버튼을 누르면 "삭제한 댓글은 다시 볼 수 없어요"라는 확인창이 뜨고, 확인하면 `DELETE /comments/{commentId}` 가 호출되어 화면에서 사라집니다(데이터베이스에는 소프트 삭제로 남습니다).

4-2. 입력창 사용성 개선

초기 구현은 한 줄짜리 입력창(`<input>`)이었는데, 사용자 피드백에 따라 다음과 같이 개선했습니다.

1. **여러 줄 입력 가능** — 한 줄 입력창을 **여러 줄 입력창(textarea)**으로 교체했습니다.
2. **최대 50줄 제한** — 사용자가 직접 타이핑하거나 긴 글을 붙여넣더라도, 51번째 줄부터는 자동으로 잘려서 저장되지 않습니다.
3. **Enter키로 제출되지 않음** — 예전에는(또는 흔한 게시판 관례상) Enter를 누르면 바로 댓글이 등록되는 경우가 많은데, 여기서는 그렇게 하지 않습니다. Enter는 줄바꿈으로만 동작하고, 오직 **"등록" 버튼을 눌러야만** 댓글이 실제로 전송됩니다. 실수로 Enter를 눌러 미완성 댓글이 올라가는 것을 막기 위한 결정입니다.

4-3. 댓글 좋아요 신규 기능

각 댓글 아래에 하트 아이콘과 좋아요 개수가 표시됩니다.

1. 로그인하지 않은 사용자에게는 하트가 회색으로 보이기만 하고 눌러지 않습니다.
2. 로그인한 사용자가 하트를 누르면 `PUT /comments/{commentId}/reactions/LIKE` 가 호출됩니다.
3. 서버는 `community_comment_reaction` 테이블에서 "이 사용자가 이미 이 댓글에 좋아요를 눌렀는지"를 확인합니다. 이미 눌렀다면 그 기록을 지우고(좋아요 취소), 안 눌렀다면 새로 추가합니다(좋아요 커기).
4. 서버가 최신 좋아요 개수를 다시 세어 응답하면, 화면은 하트 색(눌렀으면 빨간 하트, 아니면 회색)과 숫자를 즉시 갱신합니다.

4-4. 댓글 수정 신규 기능

본인이 쓴 댓글에는 연필 아이콘(수정)과 휴지통 아이콘(삭제)이 함께 보입니다.

1. 연필 아이콘을 누르면 댓글 내용이 그 자리에서 바로 편집 가능한 입력창으로 바뀝니다(별도 팝업 없이 인라인 수정).
2. 수정 중에는 취소/저장 버튼만 보이며, 여기서도 최대 50줄 제한이 그대로 적용됩니다.
3. 저장을 누르면 `PATCH /comments/{commentId}` 가 호출되어 서버가 작성자 본인이 맞는지 다시 한번 확인한 뒤 내용과 수정 시각(`updated_at`)을 갱신합니다.
4. 저장이 성공하면 편집 모드가 닫히고, 댓글 옆에 **"(수정됨)"** 표시가 나타납니다. 저장이 실패하면 편집 모드가 그대로 유지되고 에러 메시지가 보입니다.

 왜 본인 댓글에만 버튼이 보이나요

화면이 스스로 "이 댓글 작성자가 나인지" 계산하지 않습니다. 서버가 댓글 목록을 내려줄 때마다 로그인한 사용자를 기준으로 각 댓글에 `isMine` (내 댓글인가) 값을 함께 계산해서 내려주고, 화면은 그 값을 그대로 사용합니다. 서버가 수정/삭제 요청을 받을 때도 작성자 본인인지 다시 한번 검증하기 때문에, 설정 화면 쪽에서 버튼이 잘못 보이더라도 실제로 남의 댓글을 고치거나 지울 수는 없습니다.

5. 이번 세션 작업 순서 정리

무엇을 요청했고, 어떤 순서로 만들어졌는지

- 1 댓글 기능 백엔드 구현 — 댓글 목록 조회, 작성, 삭제 3개 API와 `community_comment` 테이블 기반 로직을 처음부터 구현했습니다.
- 2 "댓글쓰기 버튼이 안 보인다"는 피드백 — 확인해보니 백엔드만 만들어져 있고 화면(상세 페이지)에는 연결이 안 되어 있었습니다.
- 3 화면 연동 완료 — 댓글 목록/작성/삭제를 상세 페이지에 실제로 붙여서 눈으로 보고 쓸 수 있게 만들었습니다.
- 4 입력창 사용성 개선 요청 — 한 줄 입력창을 여러 줄(textarea)로, 최대 50줄 제한 추가, Enter 제출 제거(버튼으로만 제출)를 반영했습니다.
- 5 좋아요 · 수정 기능 추가 결정 — "댓글에 좋아요나 수정 기능도 가능한가?"라는 질문에 두 기능 모두 데이터베이스 구조 변경(신규 테이블 1개 + 컬럼 1개 추가)이 필요하다는 점을 안내했고, 사용자가 두 기능 모두 지금 구현하기로 선택했습니다.
- 6 좋아요 · 수정 기능 구현 — 마이그레이션 2개(`V20` , `V21`), 백엔드 로직, 프론트 화면(하트 버튼, 연필 아이콘, 인라인 편집)까지 전 구간을 구현했습니다.
- 7 사용자 컴퓨터로 전달 완료 — 만든 파일 18개(백엔드 11개 + 프론트 7개)를 모두 사용자의 로컬 저장소에 반영했습니다. **아직 git 커밋은 하지 않았습니다.**

6. 다음에 확인해야 할 것

이 작업 환경에서는 실행할 수 없었던 검증 항목들

⚠ 왜 자동으로 테스트를 못 돌렸나요

이 작업이 이루어진 환경(클라우드 작업공간과 사용자 컴퓨터에 연결된 원격 실행 도구)은 보안 정책상 외부 패키지 저장소(Maven Central 등) 접속이 막혀 있어 백엔드 빌드(`./gradlew test`)를 돌릴 수 없었습니다. 프론트엔드도 사용자 로컬의 `node_modules` (설치된 라이브러리 폴더) 일부가 손상되어 있어 `npm run lint` 를 돌릴 수 없었습니다. 그래서 코드는 문법 구조를 꼼꼼히 검토하는 방식으로만 확인했고, 실제 실행 검증은 아래 항목을 사용자가 직접 해주셔야 합니다.

백엔드

- ☐ `./gradlew test` 실행 — 특히 새 마이그레이션(V20, V21)이 정상 적용되는지, 댓글 관련 단위/통합 테스트가 전부 통과하는지
- ☐ 데이터베이스에 `community_comment_reaction` 테이블과 `community_comment.updated_at` 컬럼이 실제로 생성되었는지 확인

프론트엔드

- ☐ `npm install` 로 손상된 `node_modules` 복구
- ☐ `npm run lint` , 가능하면 `npm run build` 실행

브라우저에서 직접 눈으로 확인

- ☐ 댓글 입력창이 여러 줄(textarea)로 보이는지
- ☐ 50줄 넘게 텍스트를 붙여넣었을 때 자동으로 잘리는지
- ☐ Enter를 눌렀을 때 줄바꿈만 되고 댓글이 등록되지 않는지, "등록" 버튼을 눌러야만 등록되는지
- ☐ 하트(좋아요) 버튼을 눌렀을 때 색과 숫자가 바로 바뀌는지, 다시 누르면 취소되는지
- ☐ 본인이 쓴 댓글에만 연필/휴지통 아이콘이 보이고 다른 사람 댓글에는 안 보이는지
- ☐ 연필 아이콘으로 수정 후 "(수정됨)" 표시가 뜨는지

기타

- ☐ 위 항목이 모두 확인되면 `git commit` 으로 반영(현재는 파일만 반영되어 있고 커밋 전 상태)
- ☐ `docs/community-api-contract.md` 에 이번에 추가된 댓글 수정/좋아요 API를 반영할지 여부 결정

이 문서는 KDT_TravelPlanner 프로젝트의 커뮤니티 댓글 기능 개발 세션 내용을 바탕으로 자동 생성되었습니다.